

PREDICTING ASSET RETURNS: MACHINE LEARNING WITH FREE DATA

P. EDWARD HERMAN

ABSTRACT. In this paper, we model expected stock returns from a combination of various freely available internet data sources. When predicting expected stock returns, the go-to data for new ideas and benchmarking comes from CRSP and Compustat. However, this data requires a license, which the common retail investor may not have. We aim to investigate what gains can be had in prediction power by using publicly available data and machine learning.

Secondary, we aim to understand which publicly-derived features are important in making predictions of asset returns. Locally, the model pinpoints the features most important in predicting returns on a single equity. Traditionally, the model also determines significant features on a global, aggregate level.

1. INTRODUCTION

It is not easy for the common retail investors to model and predict asset returns on their own. One challenge is not being able to access complete quarterly accounting data. Often, freely available data is limited to 3-5 years. Likewise, time-series data is also not fully available. Making the public data useful for modeling presents another challenge. Different sources have their own methods of collecting and displaying the same data. After overcoming these challenges, the most important question becomes: can non-trivial prediction power for asset returns be realized? We answer this question by collecting data from various public resources and then, by applying machine learning as a modeling device.

In this paper, we try to predict expected total stock returns for the Russell 3000 Index with various length investment windows. This return includes capital appreciation and dividends, but no buybacks. The features we build for modeling can be seen in financial academic literature such as Price-to-Book ratio, Enterprise Value, or the esoteric Piotroski F-Score.

Unlike the works of [FF1],[FF2], we do not view our data as a time series for two reasons. One, is the lack of data. The advantage of working with the academic-licensed Compustat data is that it goes back over 60 years. Given we have 3-5 years of data, applying cross-sectional time series regression to each months return would not lead to strong significance tests.

Two, is that we are not concerned with finding risk premia, rather the aim is to look at raw prediction power. We reference [DGK] as a paper using machine learning to predict equity returns that also does not use a cross-sectional model for every time period. In this paper they would have one row per month of fundamental data for each equity. In contrast to [DGK], we create one row of data for each equity, where we build out time trended features. For example, for a given equity for a four year time period, we “flatten” the data into a single row of data. This gives us as independent variables: one equity per row and one column per engineered feature as input to our model. We now need an appropriate target variable.

For our target return variable, we look at several long range time windows with respect to the time period of the independent variables. Specifically, we have certain knowledge on the last day of the annual year and predict expected total stock returns 1 year plus X month(s) out where X ranges from 0 to 4. We expect that as the time window lengthens fundamental data, rather than just pure pricing data, plays a more important role in determining returns. We also observe how our model rank ordering power increases or decreases as we lengthen the window. Rank ordering power is more a view of the machine learning model itself rather than a financial metric. However, strong rank ordering power leads to stronger prediction power and therefore stronger financial results with proper portfolio building.

An additional reason to choose such a long window is to see if the edge at this longer window of time persists. With so many competitors in the financial market trying to create alpha at short time periods (seconds, days, months), is there an advantage for a patient retail investor? In other words, can we predict total expected returns at longer periods of time?

1.1. Our Contributions. Choosing the correct metric is important in optimizing a machine learning model. To predict equity returns, we introduce Gini, a rank ordering metric for a continuous target variable. Analogous to using Root Mean Square Error (RMSE), this metric is a way to tune models for a continuous target variable. The metric also relates to AUC (Area under the ROC curve), a metric for understanding the rank ordering of a classification modeling. For example, we could convert our returns target variable into a binary variable defined as:

$$(1.1) \quad \text{Binary_Return_Target} = \begin{cases} 1 & \text{if Return} > 20\% \\ 0 & \text{otherwise.} \end{cases}$$

Then, we could tune such a model with that target variable by maximizing AUC. Gini is AUC but extended to a continuous variable and without making a choice of cutoff as we did at 20%. We discuss this more in Section 4.5.

We show the Gini metric alongside the more commonly used R^2 for holdout data. The R^2 is used as a performance metric for models in recent asset prediction papers using machine learning [DGK], [Zoo]. It is not clear if our R^2 metric can be compared to their R^2 as we are using different investment windows. As well, they use a time-series versus our flattened time data setup.

Outside the December 2018 holdout data, our Gini/ R^2 seems steady as we lengthen the time investment window. It would be interesting to look at the distribution of Gini as we continue extending the investing window. Our rank ordering does drop off from predicting in-time samples to our holdout as we would expect, but we see from a portfolio performance in Section 5 that can still get Sharpe Ratios over 5.

Machine learning models often improve in performance with more features. This is especially true with our gradient boosting trees algorithm: XGBoost. This algorithm is quite robust to overfitting. One idea we introduce, which leads to many different features, comes from Discounted Cash Flow analysis (DCF). To get DCF, an analyst obtains growth, free cash flow, discount rate. These quantities change as time goes on (present vs. continuing value, 2-Stage DCF, etc). The DCF model is subjective to each analyst. In our model, we automate the construction of many DCF models that we use as features.

One parameter we perturb within the DCF model features is varying the number of transition periods. In particular, we create DCF features with 2 and 3 transition periods to have

a richer feature space. We do see that some of the DCF features we create this way are robust in rank ordering equity returns across multiple investment windows. One of these features is:

$$DCF_morn_YOY_damo_growth_c_5_decay_ebit_75_roic_80_4.$$

This feature uses fundamental data from year 4 (.4 suffix above) and has two transition different periods, a present and continuing value. The present value calculation lasts 5 years (*c.5* above), using as growth Morningstar’s ‘Operating Income growth year over year, Damodaran’s sector ROIC and WACC. The continuing value decays the growth variable from the present value by 75% and decays the ROIC by 80%. We did not know these parameters were a suitable choice to make a good DCF feature. The model selected this particular DCF by its ability to improve the rank ordering of the expected returns across our whole equity universe, nothing else. A more important variable than the above DCF feature is:

$$DCF_damo_sector_growth_10Y_damo_growth_c_10_decay_ebit_10_ROIC_without_leases_20_per_share_pct_change_last_1_years_from_2$$

which is the percentage change in a DCF feature from year 1 to year 2 with 10 years of time splitting the present versus continuing value. The “_per.share” divides the total value by diluted shares. It uses Damodaran’s EBIT sector growth as well as his sector ROIC without leases. Growth decays to 10% of its original value and the chosen ROIC decays to 20% of its value. We explain in detail the many different DCF features we build in Section 3.3.

We stress a single machine learning algorithm: XGBoost. This algorithm also allows us to have missing values in the data. For example, the ability to include missing values allows us to compare recently IPOs with older companies even though the former has less performance data. From the vantage point of XGBoost, the lack of a feature for an equity is as important as having a value. Likewise, we can include financial services and insurance companies in our equity universe, which have extra features. For example, these financial companies have a field “Deposits” in Morningstar’s data the majority of equities do not have. Placing a missing value in “Deposits” for non-financial equities allows us to extract signal from this feature.

We show how XGBoost has a macro as well as a micro feature importance. The macro importance, Gain, allows us to say what are the most important features for the aggregate model. The micro importance, Shapley values, allow us to look at what particular features affect the predicted model return at the equity level. These two metrics of importance agree for a linear regression model. However, using a linear model loses the complicated distribution of the independent variables against the target return variable. The trade-off of using a machine learning model is one more closely retains the distribution of the variables, but that knowing micro importance does not always lead to macro importance, or vice versa.

Lastly, after feature engineering and tuning the hyperparameters of our XGBoost model, we look at economic performance on several portfolios via the Sharpe Ratio. Due to the machine learning algorithm and our preprocessing of data, we see strong a Sharpe Ratio for various holdout periods. If there was a large shift in the distribution of the data we use to train our model, holdout performance could be much different. However, this statement is true for any application we know of using machine learning in financial forecasting. To protect against such shifts in a real time deployment, the return prediction should be wrapped with a model that forecasts regime shifts using macroeconomic data. But that is not the point of this paper. We only show what prediction power we can get on currently available data. Again, we stress current data as it is the only free data we can obtain.

2. DETAILS OF PAPER: DATA COLLECTION

The list of equities used in this paper from the Russell 3000 is from the holdings in *iShares Russell 3000 ETF IWV* [Ishares]. This may not be all of the Russell 3000 due to the management’s choice, but includes nearly all of them. We also use their field, Industry Group, as a way to get the sector of the equity.

2.1. Morningstar. From the tickers above, we grab fundamental data from Morningstar [Morn] from 2013-2018. These include, for example, the balance sheet, standard ratios, and financial health. For these, we only use annual data. While rolling annual data by quarters would be preferable, we could not get such data easily from Morningstar.

2.2. Yahoo Finance. We grab historic prices and volume from Yahoo Finance [Yahoo]. This allows us to produce momentum features for both price and volume. As well, it gives us the ability to produce historical covariance matrices for choices in portfolio management.

As mentioned, we use Morningstar for balance sheet data. However, as stocks update their 10K/10Q at different times, not all annual data is available for all Russell 3000 companies at a fixed time of pull. To remedy this, we use Yahoo Finance which quarterly updates the balance sheet. For example, if Morningstar has not updated their 2018 financial year data for Company X and Yahoo has all of the quarterly data for 2018, we construct annual data for Company X from Yahoo and sub it in.

Remark. We also created from Yahoo Finance data a model not discussed in this paper: an analysts or holders view of the market. In that model, we use analysts views of expected earnings, revenue, and price, as well as holding percentages for insider, institutions, and funds. The data is not amenable to our main model as the data is not trended. By that, we mean Yahoo does not have trended holder information.

With an analyst view model, we understand which stocks are the most underpriced from the models view (smallest residual) based on the number of analysts who follow the stock with an overall BUY suggestion. We can also seek to understand who owns the most of these undervalued stocks. Conversely, we can answer questions such as: what is the average analyst view of the holdings of a particular fund? Which fund has the most extreme residuals from the model’s perspective?

2.3. Aswath Damodaran Archive Data. Professor Aswath Damodaran [Damo] keeps archived data on sectors going back as early as 1999. His tables include yearly sector beta, cost of capital, expected revenue/earnings growth, and tax rates. We join this sector data to the more granular data from Morningstar and Yahoo.

Specifically, we use the following tables:

- (1) EVA(Economic Value Added) and Equity EVA by Industry [DamoEVA]
- (2) Fundamental Growth Rate in EPS by Industry
- (3) Operating Lease adjusted values for debt, returns and earnings
- (4) Price and Value to Book Ratios and ROE by Industry Sector
- (5) PE Ratios, PEG Ratios and Expected Growth Rates by Industry Sector
- (6) Costs of Capital by Industry
- (7) Working Capital Requirements by Industry

3. PREPROCESSING /FEATURE ENGINEERING

With the data from Morningstar, Yahoo, and Damodaran, we build more features. For each of these three sources, we call their starting features with no preprocessing: *basic Y*

features for

$$Y \in [\text{“Morningstar”}, \text{“Yahoo”}, \text{“Damodaran”}].$$

The main features we create are:

- (1) Normalizations
- (2) Metrics from academic finance papers and used on Gurufocus [Guru],
- (3) ROIC/DCF analysis. The latter we borrow heavily from [Valu].

We describe these enhancements in detail now.

3.1. Normalizations. We follow, and are motivated by, the normalization of fundamentals as in [Euc]. In particular, they normalize by market capitalization. As we reiterate from the introduction, we do not limit ourselves in the number of variables and so we append other normalizations. Formally, for each item x from the balance, income, and cash sheets in *basic Morningstar features*, we create features $x \div z$ for

$$z \in [1, \text{“Market Cap”}, \text{“Revenue”}, \text{“Enterprise Value”}].$$

For each feature x we get 3 more additional features. Note:, dividing by 1 is our way of including the original feature itself.

Remark. We had less choices for market capitalization/enterprise value due to only having balance sheet data at the end of the calendar year (and not quarterly rolling data as one would have in CRSP or Compustat data). Ideally, we would like the yearly balance sheet data for each equity to be available after the end of each respective calendar year. Usually, it is available after the first quarter has ended of the next calendar year. Rather than time that specific day, we just use the price at that respective year’s end price and have less precision in model performance. Using this time period should be like investing at the end of the next calendar year’s Q1, but defining the market capitalization by the price at the prior calendar’s end.

For comparison in [FF1], the market capitalization in their book value to market capitalization ratio is chosen at calendar year end as we do. On the other hand, they use the price in June of the next year for the purpose of segmenting the “size” premium in the market capitalization itself.

One can ask if using more up-to-date prices for the above ratios improves predicting returns? Indeed, the paper [AF] explores how performance of market returns vary with the time chosen for price in the book-to-market capitalization ratio. They create a new value risk premium, HML-DEV, rather than the original HML from [FF1] by using a more recent price or market cap in the book-to-market capitalization ratio. We will see this factor again in Section 5.3 when we decompose a specific portfolio of ours into canonical risk factors.

3.2. The collected metrics of Gurufocus. There have been many academic papers that define factors (or metrics) that aid in understanding the risk/reward tradeoff. We are interested in which of these metrics have greatest rank ordering power for predicting returns. However not knowing which features are the best, we try to include as many as we can in our training data. The Gurufocus website [Guru] has collected a diverse set of metrics found across academic papers investigating predicting asset returns. We take the definitions there and build our own metrics.

Some examples of metrics we use are:

- (1) Piotroski F-Score
- (2) Altman Z-Score
- (3) Beneish M-Score

- (4) Peter Lynch Fair value
- (5) Earnings Power
- (6) Graham number

See [Guru] for definitions of the above metrics.

3.3. Valuation/ROIC. There are many ways of exploring discounted cash flow (DCF) analysis. It is also easy to make mistakes in DCF across various factors: growth, horizon, cost of capital [Maubdcf]. To avoid missteps with a single DCF model, we add many variants of DCF models each as their own engineered feature. We take the ideas from [Valu] of the “key value driver formula”:

$$(3.1) \quad KDF(\text{NOPLAT}, \text{growth}, \text{ROIC}, \text{WACC}) := \frac{\text{NOPLAT} * (1 - \frac{\text{growth}}{\text{ROIC}})}{\text{WACC} - \text{growth}}.$$

We discuss feature engineering for each one of these terms: NOPLAT, growth, ROIC, and WACC.

3.3.1. NOPLAT. We have seen the terms above defined in different ways. For example, we have seen NOPLAT defined as: “EBIT - Income taxes coming from core operations.” Conversely, [Valu] describes NOPLAT as adding to EBIT the capitalization of: leases, research and development, and branding costs. The amortization of these periods of capitalization can all be different thus making the forecasting more difficult. As there seems to be a plethora of NOPLAT definitions that could be of use in separating risk, we cannot, with certainty, ignore any. Therefore, we build various

$$\text{NOPLAT}_{i,j,k}$$

with $i, j, k \in \{0, 1\}$. Here i (resp. j, k) is 1 if we capitalize lease (resp. $j = 1$ if we capitalize R&D, resp. $k = 1$ if we capitalize SG&A). As our data is limited, we never have the amortization schedule of the capitalization of leases last more than 4 years. If we had more data we could try exploring various length amortization schedules as a hyperparameter to the modeling process. For example, we could put various length schedules as features into the training-set (e.g. 4,5,6,.. year schedules independently for each capitalized object). We understand that there should be a correct amortization schedule for each capitalized object for each equity in our universe, but our goal is more mundane. Our aim is to create features that segment well (maximize information value). Some features we create will be good at separating returns, others will not.

3.3.2. growth. By definition, growth here is annual growth of NOPLAT. As mentioned in the prior section, NOPLAT is not always well-defined. This results in growth also being ambiguous. From our Morningstar [Morn] and sector data of Damodaran [Damo], we have several choices for a growth variable. These are:

- (1) Morningstar
 - (a) Operating Income growth Year over Year
 - (b) Operating Income growth Three Year Average
 - (c) Operating Income growth Five Year Average
 - (d) Operating Income growth Ten Year Average
- (2) Damodaran
 - (a) Expected Growth in EBIT.

We realize there often missing values for equities that went public later. We rely on the XGBoost algorithm to deal with this problem, see Section 4.3.

3.3.3. *Return on Invested Capital - ROIC.* We have multiple ROIC definitions dependent upon NOPLAT above as well as Invested Capital. For each $NOPLAT_{i,j,k}$ from 3.3.1, we have a respective and analogously defined $Invested_Capital_{i,j,k}$ which allows us to define

$$ROIC_{i,j,k} := \frac{NOPLAT_{i,j,k}}{Invested_Capital_{i,j,k}}.$$

We also use some sector ROIC's defined by Damodaran.

3.3.4. *WACC.* Analogous to the prior sections, we have sector weighted average cost of capital, which we label “Sector WACC” from [Damo]. Moreso, any term starting with *Sector*... comes from the sector definitions of [Damo]. As well, we construct a “WACC from scratch” for a specific equity following [Valu] as:

$$(3.2) \quad \text{WACC from scratch} := \left(\frac{(\text{Cost of debt from scratch}) \times (1 - \text{Sector Tax Rate})}{\text{Enterprise value}} \right) \times (\text{Adjusted total debt}) + (\text{Cost of equity from levered beta}) \times (\text{Market Cap}).$$

Where for the above equation we have:

$$\text{Adjusted total debt} = \text{Total non-current liabilities} + \text{Total current liabilities}$$

$$\text{Enterprise value} = \text{Adjusted total debt} + \text{Market Cap}$$

$$\text{Cost of debt from scratch} = \frac{\text{Interest Expense}}{\text{Adjusted total debt}}$$

$$\text{Cost of equity from levered beta} = (\text{Beta levered}) \times (\text{Risk Premium for Equity}) + (\text{Long Term Treasury bond rate})$$

$$\text{Beta levered} = (\text{Sector beta}) \times \left(1 + \frac{\text{Adjusted total debt}}{\text{Market Cap}} \right).$$

The terms “Total non current liabilities”, “Total current liabilities”, and “Interest Expense” are specific to each equity and come from [Morn].

Note our “WACC from scratch” uses the “Sector beta” as the beta in the cost of equity calculation. This agrees with the WACC calculation in [Valu][“Estimating the Cost of Capital”]. The authors there start with the sector beta as it has lower variance over time than a beta at the level of the equity itself. Therefore, the “WACC from scratch” should have lower variance than a direct calculation of an equity beta.

3.3.5. *Putting NOPLAT, growth, ROIC, and WACC together.* The goal is to get different views of DCF from the various inputs to the “key value driver formula” (3.1) from the sub-sections above. Namely, for each equity in our universe and each choice of definition of NOPLAT, growth, ROIC, and WACC above we will produce its own DCF feature we can use in our model. We should say we only choose the natural ROIC for the associated NOPLAT choice. For example, we plug

$$(3.3) \quad \begin{aligned} \text{NOPLAT} &= \text{NOPLAT capitalizing R\&D} \\ \text{growth} &= \text{Operating Income growth Five Year Average} \\ \text{ROIC} &= \text{ROIC capitalizing R\&D} \\ \text{WACC} &= \text{WACC from scratch} \end{aligned}$$

into the *KDF* function (3.1) above for a new feature in our model.

3.3.6. *Two and Three stage Key driver formula Feature Engineering.* The assumption built into the *KDF* features above is that growth (resp. *ROIC*) is perpetual at whatever *growth* (resp. *ROIC*) number is chosen. In [Valu], it is stressed that after some amount of years, growth reverts to a mean growth. Likewise, *ROIC* will regress to the equities *WACC*. Again, since we are uncertain how long it takes to have such regression, we create various two stage *DCF* features. As we are uncertain how much the growth and *ROIC* drop in the second stage, we also use various decays in these parameters. We define the two stage *DCF* feature:

$$(3.4) \quad KDF_{2,PV}(NOPLAT, g_A, ROIC_A, WACC, t_A) := \sum_{x=1}^{t_A} \left(NOPLAT \times \left(1 - \frac{g_A}{ROIC_A} \right) \times \frac{(1 + g_A)^x}{(1 + WACC)^x} \right),$$

$$KDF_{2,TV}(NOPLAT, g_A, g_B, ROIC_A, ROIC_B, WACC, t_A) :=$$

$$(3.5) \quad \frac{NOPLAT_B * (1 - \frac{g_B}{ROIC_B})}{(WACC - g_B) * (1 + WACC)^{t_A}}$$

where

$$\begin{aligned} t_A &= \text{number of years in the first stage} \\ g_A &= \text{expected growth rate in the first stage} \\ g_B &= \text{expected growth rate in the second stage} \\ ROIC_A &= \text{expected ROIC during the first stage} \\ ROIC_B &= \text{expected ROIC during the second stage} \\ NOPLAT_B &= NOPLAT * (1 + growth_A)^{t_A+1}. \end{aligned}$$

We can parametrize any 2-stage DCF model for a fixed equity *A* with an initial

$$[NOPLAT, ROIC, growth, WACC]$$

by a tuple

$$[t_0, growth_{decay}, ROIC_{decay}]$$

by plugging into the formula *KDF*₂ above

$$\begin{aligned} t_A &= t_0 \\ ROIC_A &= ROIC \\ NOPLAT_A &= NOPLAT \\ g_A &= growth \\ g_B &= growth_{decay} * g_A \\ ROIC_A &= ROIC \\ ROIC_B &= ROIC_{decay} * ROIC_A \\ NOPLAT_B &= NOPLAT * (1 + growth_A)^{t_A+1}. \end{aligned}$$

By varying the tuple:

$$(3.6) \quad [t_0, growth_{decay}, ROIC_{decay}]$$

across our universe of equities we can create many features each of which could be considered an alternative view of valuation. For example, a tuple could be $[3, .5, .6]$ which for a particular starting choice of

$$[NOPLAT, ROIC, growth, WACC]$$

say from (3.3) gives us a feature:

$$KDF_{2,PV}(NOPLAT, growth, ROIC, WACC, 3) + \\ KDF_{2,TV}(NOPLAT, growth, growth * (.5), ROIC * (.6), WACC, 3).$$

Remark. Note: we do not update the WACC. Ignoring an update on WACC could be incorrect, but having a custom update to WACC in addition to the update to growth and ROIC would exponentially increase the number of features when already there are many features. We see the authors of [Valu] do not update their WACC as well when splitting apart their present vs continuing value calculation either.

For each starting choice as in (3.3) in our code, we can build many new features by adjusting our decay-tuple (3.6). One can consider this as a double loop with an outer loop of starting tuples like (3.3) and an inner loop over decay-tuples as in (3.6). We try to choose the inner loop of decay-tuples that make sense to us while the outer loop of starting tuples are chosen from historic data as in (3.3).

We have the analogous three stage model using the tuple of parameters:

$$[[t_{A,1}, t_{A,2}], [growth_{decay,1}, growth_{decay,2}], [ROIC_{decay,1}, ROIC_{decay,2}]].$$

It is completely analogous to the decay-tuple for the 2-stage model (3.6). But now the starting *growth* and *ROIC* is used for time period: $[0, t_{A,1}]$. While

$$[growth * growth_{decay,1}, ROIC * ROIC_{decay,1}]$$

is used for time period:

$$[t_{A,1}, t_{A,2}].$$

Lastly,

$$[growth * growth_{decay,2}, ROIC * ROIC_{decay,2}]$$

is used for terminal period: $[t_{A,2}, \dots]$.

For example, with the tuple $[[[5, 10], [.7, .4], [.6, .3]]]$, we have the following three-stage model. The starting *growth* and *ROIC* is used for time period: $[0, 5]$. While

$$[growth * (.7), ROIC * (.4)]$$

is used for time period: $[5, 10]$. Lastly,

$$[growth * (.6), ROIC * (.3)]$$

is used for terminal period: $[10, \dots]$. It should be clear to the reader how large the feature space could grow with different tuple choices.

Remark. What are good tuple choices? One answer is to look at different scenarios an equity could go through as time passes. For example, one could have a “conservative” valuation where decay of ROIC to WACC is very quick and growth converges to the sector mean. Another example is a “best case” scenario where equities sustain their high ROIC and growth for long periods of time. Each of these scenarios can be converted into tuples as above.

3.4. Trended data. So far we have discussed data or features engineering ideas from Morningstar, Yahoo, Damodaran, and DCF/Valuation theory. A natural question is to understand the average or percentage change of these features over time. Therefore, for each feature $X_{i,y}$ for a given equity i and year y , we also look at change in that variable from the past year as well as an average of that variable over some past number of years. Namely, we add as features:

$$X_{i,y}^{\text{delta}} := \frac{X_{i,y} - X_{i,y-1}}{X_{i,y-1}},$$

and for a number of years T we look at the average:

$$X_{i,y}^{\text{average},T} := \frac{1}{T} \sum_{j=0}^{T-1} X_{i,y-j}.$$

For example, from the last section we have 2 and 3 -stage DCF features, now we will have additional features that are changes and averages of those features. This is not so uncommon as we see from the aggregated collection of factors in [Zoo]. Of course, some will not be useful at all but that does not matter with our choice of algorithm [XGB].

How do we label these trended features in our model? We see some examples in Table 18. For example, for trended data of type $X_{i,y}^{\text{delta}}$ for the Damodaran’s sector feature *Expected_growth_next_5_years* starting from year 3 we have:

$$\text{Expected_growth_next_5_years_pct_change_last_1_years_from_3}.$$

The *last_1_years* says we look back only one year for a percentage change.

Remark. One could explore percentage change from 2 years back or more given one has the data, but we do not.

While, an example for $X_{i,y}^{\text{average},T}$ with the feature *Altman_Z_score* starting from year 4 is:

$$\text{Altman_Z_score_average_last_4_years_from_4}.$$

3.5. Connecting the above data sources. We mention four sources of data for ticker/sector, fundamental, and pricing: Ishares, Morningstar, Yahoo, and Damodaran’s data. As Damodaran’s data is sector only, we can join his data with Ishares *IWV* holdings via their field, “Industry Group.” The sector names from the Ishares and Damodaran’s data tables do not line up, so we use Scikit-learn’s *Tf-idf Transformer* [Tfid]. *Tf-idf*, or term-frequency times inverse document-frequency, converts a document, or in this case, a two or three word “Industry Group” into a set of numbers that are weighted by how many times a word appears in a single group versus the number of times it appears in the whole collection of Industry Groups. After predicting which Industry Group matches which sector of Damodaran, we have a pipeline from ticker symbols to sector data.

It is more straightforward to join the ticker data to Morningstar and Yahoo data as they can all be joined by the ticker symbol itself. However, if one wants to predict on very recent data there is a problem with fundamental data. As fundamental data is not updated all

ID	Name	Built	Lack data to build	Why not built	Possible Replacement
148	HXZ Profitability	0	0	Need ROE with quarter lag on Book, have similar	$ROTC_{i,j,k}$
88	Change in Financial Liabilities	1	0	N/A	adjusted_total_debt_pct_change_last_1
51	Industry-adjusted size	0	0	Can build but did not	Have Industry-adjusted price and volume
62	Liquidity	0	0	Can build but did not	N/A
74	Tax income to book income	0	0	Can build but did not	N/A
61	Illiquidity	0	0	Can build but did not	N/A
49	Industry-adjusted cash flow to price ratio	0	0	Have similar	Free_cash_flow_MC_normalized
122	Sin stocks	0	0	Have similar	Sector as a categorical variable
6	Share price	1	0	N/A	price_last_month_of_year_X
55	Advertising Expense-to-market	0	0	Have similar	Sales_General_and_administrative_MC_normalized
72	Net Operating Assets	1	0	N/A	Scaled_Net_Operating_Assets
53	Volatility of liquidity (dollar trading volume)	0	0	Can build but did not	N/A
119	Industry-adjusted change in asset turnover	0	0	Have similar	TotalAssets_Rev_normalized
140	Betting Against Beta	1	0	Sector Beta	Beta
44	Industry adjusted % change in capital expenditures	0	0	Have non-adjusted	Capital_expenditure_pct_change_last_1
147	HXZ Investment	1	0	Have non-adjusted	TotalAssets_pct_change_last_1
65	Order backlog	0	1	Compustat specific	N/A
32	Sales to price	0	0	Enterprise value normalized	Revenue_EV_normalized
31	Working capital accruals	0	0	Have similar	Change_in_working_capital
87	Change in Short-term Investments	1	0	N/A	Short_term_investments_pct_change_last_1
123	Revenue surprise	0	1	Need trended expected earnings from analysts	N/A
5	Unexpected quarterly earnings	0	1	Need trended expected earnings from analysts	N/A

TABLE 1. Most important risk factors found in a step-wise fashion ignoring time of discovery.

at the same time for all equities, our Morningstar data is incomplete. For example, we need 2019 fundamentals, but they have not been updated. If they have “Through Twelves Months,” or *TTM*, we can use that as a proxy. However, at times *TTM* is also missing. In these cases, Yahoo will have quarterly data which we then convert to Morningstar-like annual data. We go through this hard work as Morningstar has the most comprehensive data at an equity-level.

In summary, we use Morningstar data as our main source. We fill in “holes” in the data with Yahoo data. This Morningstar-Yahoo aggregation of fundamental data at an equity level is then joined back to Damodaran’s sector data via *IWV*’s “Industry Group” data.

We use Python and in particular the packages: *Pandas*, *Numpy*, *Scikit-Learn* for building out the data pipeline.

3.6. Our features verses Factors in Literature. How do the features we create compare to features or factors used in risk premia literature? To answer this question, we reference [Zoo], a work which tries to collect together all of the factors used in academic asset pricing papers. They cite 150 factors across literature. One section of the paper, *3.4 A Forward Stepwise Procedure*, discusses which of the 150 factors would be most important if all factors were created at the same time. More specifically, they start with a baseline of 4 factors in the starting model: 3 factors are from [FF1] which we discuss in Section 5.2 and a fourth is a momentum factor UMD. They then search through the other 146 factors one-by-one for the one that has the highest *t*-statistic with the other 4 baseline factors. This new factor with the highest *t*-statistic is added to the original 4 to create a new baseline set of factors. This process is then iterated creating new baselines until the next contribution to the current baseline set of factors is considered insignificant. The factors they find in order (with associated ids) are found in Table 1.

In this table, beyond the ID column, we add a binary column for whether we lack the data to build the respective feature. Also, we add columns to account for: a) if we did not build the column, why not, and b) a column that provides a replacement for the feature.

4. METHODOLOGY

The goal of this section is to explain our modeling setup. We take a supervised approach to predicting normalized equity returns. For a supervised approach, we build a function from

Label	Train/Target Month	Holdout/Target Month
Dec18	2013-2016/Dec-17	2014-2017/Dec-18
Jan19	2013-2016/Jan-18	2014-2017/Jan-19
Feb19	2013-2016/Feb-18	2014-2017/Feb-19
Mar19	2013-2016/Mar-18	2014-2017/Mar-19
Apr19	2013-2016/Apr-18	2014-2017/Apr-19

TABLE 2. Train and Holdout Periods.

independent features paired with an associated dependent target variable. The then-built function can be used to predict future input data.

This is in contrast to an unsupervised approach seen in [GKX]. Another simple example of unsupervised learning would be to cluster equities via a k-means model [KM], using their associated fundamental and engineered data mentioned in Section 3. For a prediction of a future return of a particular equity, we would get the average historical return of the cluster “closest” to that equities fundamental and engineered data. This unsupervised approach may not be successful from an investing standpoint, but can help model returns without an explicit target variable.

4.1. Target Variable. Our target variable is the Sharpe Ratio for a portfolio consisting of a single equity. Our working definition for Sharpe Ratio is a quotient with numerator capital appreciation of the equity itself plus dividend yield minus the ten year bond yield annualized appropriately. The denominator of the quotient is the standard deviation of the equity return over the same period of time. This ratio written out explicitly is:

$$(4.1) \quad \frac{\text{capital appreciation} + \text{dividend yield} - \text{ten year bond yield}}{\text{std of the equity return}}.$$

We tried modeling with capital appreciation plus dividend yield. However, the portfolio chosen by the model selected microcap companies with huge variance in returns. If we stuck with just this target variable, we expect that, with different training data and different time snapshots, the out- of- time portfolio selected by the model would not be very stable.

4.2. Time periods for data. As stressed before, the lack of free data over many years makes long term analysis difficult. We look at training data covering 4 years, 2013-2017, with a target variable a year, or more, forward in time. The out of time holdout covers the same time range but shifted a year. In other words, the holdout covers years 2014-2017 with target variable from various months in 2019.

The data has 2,353 different equities with 12,481 original plus engineered features.

We label below 5 different periods of time that, for the respective holdout data, we examine returns and rank ordering power for in Table 3.

Each of the paired training and holdout sets is Market Cap normalized by December 31 of the prior year in the respective set. For example, the training data for *Jan19* has Market Cap as of Dec 31, 2016 while the holdout set has Market Cap as of Dec 31, 2017. The Market Cap would go into features such as:

$$\text{Enterprise value, } \frac{\text{Total Assets}}{\text{Market Cap}}, \frac{\text{Enterprise value}}{\text{Free Cash Flow}}.$$

How does this compare to other papers time frames for predicting equity returns? In [FF2], the authors use for a given year $t - 1$ the year-end as we do for their Market cap as well as their Book value to Market Cap. Their “target variable” is the return from July of year t to June of year $t + 1$. They have 28 different years of such data; we do not.

The work [FF2] is similar to our current work in that they look one year out for returns from a given point. They look out one year uniformly for all 28 years above, while we move our length of time for our target out further for different holdout sets. Their accounting or fundamental data is *only* from the prior year. We instead take 4 years worth of accounting data for each equity which we then also convert into trended data, see Section 3.4.

In [DGK], they use machine learning with monthly returns as their target variable. They train with accounting and macro data that is appropriately lagged compared to the respective monthly return used in the regression. However, like [FF2] their independent features come from only the prior year.

4.3. Modeling Algorithm: XGBoost. We choose a popular machine learning algorithm XGBoost [XGB] for our study. XGBoost is a gradient tree boosting algorithm. More simply, the algorithm starts with building a decision tree where each successive node in the tree, and the respective rows data residing in it, has a more homogeneous target variable. Whatever leaf, or final node, of the decision tree a particular instance of data falls into leads to a prediction of the algorithm on the respective target. A second new tree is then built on the residuals. Or, the new target variable for that second decision tree is the original target variable minus the starting tree’s prediction. This idea of modeling residuals of the prior tree in the sequence can be then extended inductively to an arbitrary amount of trees. One then gets a final prediction for the original target variable by a weighted sum of the predictions of all the trees in this iterative process. The original idea of boosting came from [Brei] and a good introduction to boosting is here [M].

4.4. Missing values in the data. An important advantage of using XGBoost over other algorithms is that it makes smart use of missing values in the data. Specifically, at each node the algorithm determines a preferred branching inside the decision tree for instances with missing values. In other algorithms such as linear regression or neural networks, one is forced to choose an imputation (mean/median, etc.. of a feature) that could lead to bias. In our experience, models tend to outperform on out-of-sample data by just letting XGBoost determine how to handle missing values rather than leaving the choice of imputation to the end-user.

With respect to our own data, missing values are prevalent. For example, we include younger companies who do not have a sector 5 or 10 year-over-year operating income growth as seen in Morningstar’s data. We expect the algorithm will direct those particular young companies into the appropriate subsequent node such that these latter nodes are more homogeneous in the target variable.

4.5. Finding the best Model: Parameter tuning. As with most machine learning algorithms, there are extra parameters beyond what is in the data itself. These outside parameters, called hyperparameters in machine learning literature, are needed to be optimized for best performance on data outside the training data. To tune hyperparameters, we use 5-fold cross validation on in-time data. We look for the set of hyperparameters that performs best on average across the in-time but out-of-sample training data coming from the cross validation. In our case, the best performance is looking at maximizing Gini or Gini Coefficient

[Gini]. The Gini coefficient is a generalization of AUC from a binary to a continuous target variable. The higher the Gini, the better the rank ordering power of the model.

As mentioned before, we are looking at multiple training sets each with their own holdout set. In theory, we should tune each training set’s hyperparameters. As we saw comparable performance across all train sets for comparable hyperparameter sets, we choose a single best set of hyperparameters for all the train sets in Table 2. Therefore, the results in 5 come from the same choice of hyperparameters. Fixing this singular choice of hyperparameters simplifies our presentation.

A more appropriate way to tune these hyperparameters would be to have a validation set of data outside of the time period of training as well as the holdout data, but as we are limited in our data. We choose parameters with in-time but out-of-sample data. For example, we could truncate our training data period from 2013-2015, choosing 2016 to be the validation set and our holdout be 2017-2019. However, then the time between the end of train set and the holdout set would be 2 years.

The XGBoost algorithm has many parameters that can be finetuned. We limit our study to ones, in our experience, that have the most impact on improving the performance on out of time data. These parameters are:

- (1) Number of trees
- (2) Eta, step size used for preventing overfitting
- (3) Max depth of trees, deeper trees make model more complex and more likely to overfit
- (4) Column sample by tree, sub-sample ratio of columns when constructing each tree
- (5) Lambda, L2 regularization term on weights. Increasing this value will make model more conservative
- (6) Alpha, L1 regularization term on weights. Increasing this value will make model more conservative.
- (7) Gamma, Minimum loss reduction required to make a further partition on a leaf node of the tree. Increasing this value will make model more conservative.

As we have so many features going into the model, we do not want to overfit to noise. Therefore, regularization parameters have to be very strong. Another sign of the model being overfit, or of data leakage, is that one feature dominates the others in importance. With our final chosen selection of hyperparameters, no feature makes up more than 2% of the total importance. Now, we demonstrate how XGBoost has an intuitive way of explaining feature importance at a global level and at an equity level.

4.6. Variable Importance. XGBoost has great ways to explain the importance of features used in getting the optimal model. There are three ways of getting the importance of a feature to the model: Gain, Cover, and Weight. The three are defined here:

- (1) *Gain*: the average gain across all splits the feature is used in. In other words, when a feature is used as a split at a node, gain or information value is calculated. The higher the gain the more important that feature is used in an accurate prediction.
- (2) *Cover*: the average coverage across all splits the feature is used in. This relative contribution of how many instances are moved into the next level of the tree. For example, a feature that is very specific to a sector such as for insurance companies and the feature: "Policyholder benefits and claims incurred" from [Morn]. This feature is missing for the majority of our training data universe but could be valuable for segmenting insurance companies. This feature may fall very deep in the decision tree and so there are not many instances going to the next deeper level. If so, this feature would have low cover.

- (3) *Weight*: the number of times a feature is used to split the data across all trees. We might see a feature having high weight if the feature is highly non-linear with respect to the target variable.

We only report Gain in this paper. We just want to know what features are driving the accuracy of our predictions.

One can compare these very specific importance metrics to those in [DGK]. There they look for what features that when perturbed (in their case set to 0) display the greatest drop in R^2 from the original holdout R^2 . This seems appropriate as the point in their paper was to compare different machine learning algorithms with one unifying metric. It is not immediate how to obtain Gain on a neural network. However, in our case, we are trying to look at the predictive and interpretative power of one particular algorithm, XGBoost.

Note that the importance metrics above used by XGBoost are all at a macro level. These do not immediately help explain a particular equity's predicted return. Can we understand for a particular equity how its return decomposes into a sum of returns weighted by each individual feature? If we were using linear regression, the predicted return would be calculated as the sum of the weighted returns from the individual features. Using a machine learning model, such a breakdown is not obvious as the aggregate distribution of the features, along with the target variable, is highly non-linear. The answer we seek to break down returns at an equity-level is in using Shapley values.

4.7. Shapley values. Shapley values were originally used in cooperative game theory to answer the question: how important is each player to the overall cooperation? And what payoff can he or she reasonably expect [Shapwiki]? As well, they are valuable in understanding the importance of features in linear regression when collinearity is involved.

In [LL], the authors show the connection of this game theoretic idea to interpreting machine learning models. To show this connection, we remind the reader from the previous section what we desire. For a particular equity, we want the predicted return of any machine learning model on that equity decomposed as a sum of returns each uniquely associated to a particular feature. The authors label a model that can be explained via this additive decomposition as having *additive feature attribution*.

They prove if one wants a model explainable by *additive feature attribution*, there is only one solution: Shapley values. This proof assumes three desirable properties: Local Accuracy, Missingness, and Consistency defined in [LL]. The paper goes into how to calculate these values independent of the modeling algorithm choice. However, what is important to us is that, in [ELL], an efficient implementation for decision trees was constructed by one of the authors and is now implemented in XGBoost.

Shapley values allow us to answer how much of this return for a particular equity comes from a given feature. We show some examples of how Shapley values decompose for various equities. Often it is the case that top features by the macro model importance metric, Gain, are correlated with the largest return contributors at the level of the equity.

4.8. Performance Evaluation. As we mentioned in 4.6, we use Gini to determine our best hyperparameters. We also state the Gini of each out of time sample from Table 3. To have some comparisons to other work, we also state the out of sample R^2 . This would be the same as R_{OOS}^2 in [DGK] which is:

$$R_{OOS}^2 := 1 - \frac{\sum_i (r_i - \hat{r}_i)^2}{\sum_i r_i^2}$$

Here $OOS :=$ “Out of sample” and the sum over i is over equities in the respective holdout time period from Table 3. Our sum does not depend on their time parameter t as we have a non-time varying holdout set for a respective training set.

As we would like to have some idea of variance on our Gini metric on our holdout data, we predict with five folds. Specifically, after our tuning to find the best hyperparameters, we still break our data into five folds. For each fold, we train a model on the other four folds. With the model trained on those four folds, we score the in-time, out-of-sample data (the fifth fold), but as well, we score the holdout data. This gives five different predictions for each equity in the holdout data. This allows us to get a pointwise variance for each equity return. As well, we get the mean and variance of the Gini for these five folds, which we give in Table 3.

Remark. To make this measurement across five folds more robust, we should aggregate this scoring over several randomly seeded foldings, but we did not. We could also have done a stratified folding to normalize the average return within a fold [StratCV]. One could do stratified cross validation across many areas such as sectors, fundamental ratios, etc. It is not clear what is the correct choice to appease all readers or to make the best model.

Remark. One can ask if you are using only 80% of the in-time data to score the out-of-time data isn’t that losing possible performance (Gini, R^2)? In our experience, there is not a huge drop off in performance. We are comfortable sacrificing some performance to understand what predictions are more variant than others.

5. RESULTS

As mentioned in Section 4.2, we have five time periods to look at. Each time period has an associated training period at least one year before the date of the target variable. For each period, we have a five folded prediction of returns for in-time but out-of-sample data as well as the associated single holdout period. We show the mean and standard deviation for the Gini and the mean of R^2 for those periods in Table 3.

5.1. Variable Importance. In Table 4, we report the rankings of features by Gain (see Section 4.6 for discussions on Gain). Each non-ID column is an average of the 5-folded model’s Gain as stated in Section 4.8.

5.1.1. Missing values in Importance Table. One may notice missing values in Table 4. We discuss those now. Notice ID 1, or the feature

$$E_divided_by_D_plus_E_4,$$

is missing from the top 100 variables of the *Apr19* model marked on the table by $\text{NaN}^{(1)}$. This would suggest this information is not important to this model but is important to the

Metric	In/Out Sample	Statistic	Dec18	Jan19	Feb19	Mar19	Apr19
Gini	In Sample - Out of Fold	Mean	0.434	0.473	0.509	0.465	0.446
		SD	0.030	0.032	0.044	0.035	0.031
	Holdout	Mean	0.114	0.139	0.169	0.211	0.241
		SD	0.014	0.007	0.012	0.012	0.021
R^2	Holdout	Mean	0.636	0.698	0.758	0.758	0.771

TABLE 3. Global Gini and R^2 results.

models with earlier holdout sets. This seems hard to believe. However, this is not quite true as the *trended* version

$$E_divided_by_D_plus_E_average_last_3_years_from_4$$

shows up at #21 in importance for this particular holdout. We just do not see this trended variable showing up in our top 100 variables and so denoting it as missing is not quite true.

Likewise, the same logic as above can be applied to other missing values. We superscripted the top four in terms of importance and how the trended version actually is very important to each respective model.

For $\text{NaN}^{(2)}$ the variable

$$D_divided_by_D_plus_E_average_last_3_years_from_4$$

rather than

$$D_divided_by_D_plus_E_4$$

shows up #1 in importance for *Apr19*.

For $\text{NaN}^{(3)}$, the variable

$$D_divided_by_D_plus_E_2 \text{ rather than } D_divided_by_D_plus_E_3$$

shows up as #1 variable for *Dec18*.

For $\text{NaN}^{(4)}$, the variable

$$D_divided_by_D_plus_E_average_last_3_years_from_4$$

shows up as #1 variable for *Jan19*.

Remark. What should we conclude from missing values like above in the importance table? To understand the importance of a variable we may consider ranking an “equivalence class of features”. For example, one equivalence class could be the set composed of a feature

$$D_divided_by_D_plus_E_X$$

with different times $X \in \{1, 2, 3, 4\}$, as well as its trended versions:

$$D_divided_by_D_plus_E_average_last_Y_years_from_X$$

for $Y \leq X$. We do not do that in this paper, but it may elucidate what is truly important.

5.2. Micro Importance: Shapley Values. We saw in Table 4, the importance of variables aggregated across the five folds at a global level. By global importance, we mean the features that determined the best way to group equities with similar normalized returns. In this section, we show examples of how Shapley values can give an additive decomposition of the score our model attached to a particular equity.

An additive decomposition of our model score for a given equity is very similar to decomposing a linear regression model score. For example, a Fama-French three-factor model score r is an additive decomposition of four sub-scores plus a noise-term (alpha):

$$r = R_f + \beta_1(R_m - R_f) + \beta_2 \cdot SMB + \beta_3 \cdot HML + \alpha.$$

Here r is the equity’s model score, R_f is the risk-free return rate, and R_m is the return of the equity. The β_i are the fitted coefficients from Ordinary Least Squares (OLS). SMB stands for “Small [Market Capitalization] Minus Big” and HML for “High [Book-to-Market Capitalization ratio] Minus Low;” they measure the historic excess returns of small over big

ID	Dec18	Jan19	Feb19	Mar19	Apr19	AVE
1	4.0	4.0	1.0	2.0	NaN ⁽¹⁾	2.75
2	3.0	9.0	14.0	8.0	5.0	7.80
3	9.0	13.0	23.0	10.0	NaN ⁽²⁾	13.75
4	2.0	18.0	7.0	14.0	34.0	15.00
5	12.0	20.0	24.0	26.0	11.0	18.60
6	14.0	15.0	31.0	15.0	21.0	19.20
7	33.0	22.0	22.0	20.0	19.0	23.20
8	86.0	17.0	8.0	9.0	13.0	26.60
9	30.0	30.0	36.0	23.0	43.0	32.40
10	43.0	31.0	34.0	34.0	29.0	34.20
11	41.0	37.0	50.0	36.0	24.0	37.60
12	15.0	45.0	12.0	11.0	114.0	39.40
13	26.0	48.0	53.0	55.0	25.0	41.40
14	40.0	28.0	45.0	43.0	64.0	44.00
15	37.0	76.0	68.0	33.0	30.0	48.80
16	46.0	51.0	66.0	44.0	40.0	49.40
17	188.0	38.0	19.0	13.0	18.0	55.20
18	88.0	61.0	43.0	72.0	23.0	57.40
19	NaN ⁽³⁾	NaN ⁽⁴⁾	21.0	155.0	3.0	59.67
20	165.0	85.0	11.0	28.0	35.0	64.80
21	308.0	11.0	15.0	19.0	10.0	72.60
22	79.0	56.0	64.0	97.0	74.0	74.00
23	199.0	7.0	2.0	124.0	51.0	76.60
24	137.0	59.0	86.0	66.0	48.0	79.20
25	303.0	6.0	27.0	4.0	66.0	81.20
26	111.0	94.0	113.0	74.0	50.0	88.40
27	75.0	73.0	102.0	96.0	150.0	99.20
28	68.0	147.0	73.0	83.0	134.0	101.00
29	71.0	43.0	104.0	109.0	187.0	102.80
30	24.0	NaN	NaN	NaN	202.0	113.00
31	234.0	108.0	65.0	139.0	20.0	113.20
32	45.0	107.0	257.0	103.0	70.0	116.40
33	109.0	464.0	46.0	46.0	31.0	139.20
34	63.0	524.0	110.0	42.0	26.0	153.00
35	121.0	209.0	55.0	235.0	NaN	155.00
36	97.0	113.0	156.0	106.0	331.0	160.60
37	54.0	519.0	76.0	95.0	106.0	170.00
38	361.0	96.0	135.0	152.0	116.0	172.00
39	418.0	121.0	143.0	67.0	146.0	179.00
40	301.0	138.0	112.0	82.0	267.0	180.00
41	27.0	53.0	100.0	401.0	322.0	180.60
42	74.0	600.0	108.0	73.0	85.0	188.00
43	775.0	87.0	52.0	32.0	27.0	194.60
44	100.0	533.0	117.0	120.0	175.0	209.00
45	60.0	50.0	94.0	530.0	332.0	213.20
46	69.0	689.0	148.0	48.0	131.0	217.00
47	91.0	111.0	496.0	217.0	170.0	217.00
48	217.0	100.0	601.0	85.0	82.0	217.00
49	785.0	322.0	28.0	29.0	38.0	240.40
50	154.0	433.0	203.0	NaN	207.0	249.25
51	83.0	916.0	118.0	81.0	92.0	258.00
52	51.0	42.0	105.0	456.0	673.0	265.40
53	58.0	92.0	138.0	193.0	863.0	268.80
54	450.0	NaN	NaN	94.0	NaN	272.00
55	551.0	354.0	47.0	299.0	119.0	274.00
56	92.0	777.0	269.0	129.0	208.0	295.00
57	NaN	553.0	185.0	131.0	340.0	302.25
58	206.0	428.0	NaN	NaN	NaN	317.00
59	61.0	1328.0	96.0	64.0	52.0	320.20
60	98.0	513.0	764.0	110.0	200.0	337.00
61	746.0	234.0	256.0	309.0	159.0	340.80
62	NaN	231.0	NaN	551.0	255.0	345.67
63	163.0	55.0	33.0	1456.0	55.0	352.40
64	472.0	27.0	56.0	204.0	1107.0	373.20
65	783.0	64.0	749.0	202.0	80.0	375.60
66	152.0	407.0	1051.0	278.0	47.0	387.00
67	164.0	837.0	244.0	365.0	NaN	402.50
68	NaN	NaN	NaN	190.0	635.0	412.50
69	31.0	40.0	35.0	1229.0	739.0	414.80
70	236.0	NaN	618.0	NaN	NaN	427.00
71	321.0	515.0	155.0	869.0	292.0	430.40
72	116.0	NaN	1179.0	137.0	366.0	449.50
73	269.0	599.0	120.0	65.0	1216.0	453.80
74	NaN	21.0	26.0	397.0	1382.0	456.50
75	39.0	NaN	820.0	818.0	185.0	465.50
76	NaN	228.0	42.0	1382.0	290.0	485.50
77	NaN	619.0	362.0	NaN	NaN	490.50
78	177.0	97.0	NaN	1211.0	NaN	495.00
79	391.0	241.0	1123.0	NaN	230.0	496.25
80	1239.0	1021.0	95.0	59.0	81.0	499.00
81	17.0	112.0	513.0	1375.0	NaN	504.25
82	143.0	1244.0	284.0	405.0	493.0	513.80
83	NaN	483.0	NaN	545.0	NaN	514.00
84	1271.0	1014.0	90.0	45.0	157.0	515.40
85	176.0	299.0	89.0	662.0	1366.0	518.40
86	643.0	1477.0	300.0	101.0	84.0	521.00
87	NaN	437.0	607.0	NaN	NaN	522.00
88	844.0	226.0	NaN	NaN	NaN	535.00
89	NaN	265.0	92.0	1194.0	594.0	536.25
90	18.0	74.0	1518.0	NaN	NaN	536.67
91	NaN	NaN	755.0	540.0	327.0	540.67
92	145.0	120.0	1586.0	592.0	293.0	547.20
93	310.0	NaN	NaN	NaN	785.0	547.50
94	NaN	NaN	1098.0	500.0	76.0	558.00
95	127.0	342.0	134.0	297.0	1895.0	559.00
96	NaN	NaN	NaN	295.0	852.0	573.50
97	840.0	NaN	230.0	656.0	612.0	584.50
98	87.0	990.0	1396.0	379.0	79.0	586.20
99	287.0	690.0	263.0	872.0	838.0	590.00
100	556.0	446.0	1266.0	127.0	NaN	598.75

TABLE 4. Variable Importance by holdout month. For superscripts on missing values, see Section 5.1.1.

capitalization equities and of value over growth stocks. In particular, we would find what quantile of SMB (as well as, HML) our particular equity is in to get three sub-scores:

$$[\beta_1(R_m - R_f), \beta_2 \cdot SMB, \beta_3 \cdot HML].$$

So the prior line is a function that takes in fundamental data and outputs a vector of scores. When added, these outputs equal our model score (up to a noise factor, alpha). Shapley values have this same function.

For our model scoring a particular equity, we have a sub-score vector just as the above linear regression Fama-French three-factor model. Instead of three features, we have the total column space of our model, which in this case is a vector with over twelve thousand components. The majority of them are insignificant in absolute magnitude and so we only show the top and worst twenty Shapley values. We define top (resp. worst) in that these features and their respective sub-scores contribute the most to a positive (resp. negative) return prediction.

We give several tables of Shapley values for several large cap companies. For each equity, the tables are split in two having the lefthand columns associated to top features while the righthand associated to worst features. Within each of the sides, the first feature is an ID feature which can link to a feature name on Table 18. The second feature is the sub-score itself. The third column is a standard deviation of scores of that feature ID across a universe of 98 familiar large cap equities in Table 19. This latter column allows one to ask how extreme a certain Shapley value, or sub-score, is relative to other equities' analogous values. Let us look at Mastercard from holdout set *Apr19* as an example:

Example 5.1 (Mastercard, Table 5, Table 19). *The top Shapley value for Mastercard is ID 189, or the feature EV_Invested_Capital_1, Enterprise value over Invested Capital from year 1 (year 1 in this case is 2014). Its value is .29 which is 4% of the score 7.17 seen in Table 19. Its Shapley value for this feature compared to the 98 other large caps has standard deviation of 2.892. One can interpret these statistics as that Mastercard has one of, if not, the highest Shapley values for Enterprise value over Invested Capital for year 1 of our holdout data. As well, the model connects such high ratios to high normalized returns.*

The worst Shapley value for Mastercard is ID 200 or the feature Dividends_USD_pct_change_last_1_years_from_4, the percentage change in dividends denominated in USD in our final year of our holdout set (year 4 in this case is 2017). Its value is $-.093$ which is 1.3% of the score and this ratio compared to the 98 other large caps has standard deviation of -1.163 . We see the annual dividend dropped from 79 cents to 66 cents. The model connected this drop in dividend to a negative normalized return. The SD of -1.163 of this Shapley value if, for example, it was associated to a normal distribution would say about 12% of the population had a smaller Shapley value than that.

One can go through other values on each side of the table. For example, some Shapley values have large standard deviations but are small in magnitude. This behavior just says that though the value is extreme in this particular Shapley values distribution, it is not as significant in predicting returns. We can see this in ID 167, ROC_Greenblatt_gross_PPE_1 with a standard deviation of 5.620 but a Shapley value .111. A Shapley value less than half of that of ID 200 above.

One will note by looking across the several example tables listed that different features are important to a particular equities model score. This micro level of importance should be compared to the macro importance in Table 4, which one can consider as an aggregation of the micro importance across all equities in the train set of the model.

ID	Top_Shapley_20	Std	ID	Worst_Shapley_20	Std
189	0.290	2.892	200	-0.073	-1.201
168	0.254	1.679	6	-0.063	-2.018
15	0.239	1.840	177	-0.059	-3.373
190	0.171	2.267	202	-0.047	-1.134
2	0.143	1.227	11	-0.046	0.815
251	0.135	3.128	102	-0.043	-0.325
26	0.119	1.790	215	-0.043	-2.352
159	0.115	2.222	205	-0.042	-3.760
240	0.112	2.587	226	-0.042	-0.366
167	0.111	5.620	111	-0.040	-1.031
171	0.103	2.571	147	-0.036	-1.039
43	0.098	2.676	135	-0.035	-2.907
228	0.097	1.167	175	-0.035	-1.011
220	0.097	1.270	146	-0.034	-3.036
249	0.096	3.371	12	-0.034	-0.588
247	0.086	1.384	238	-0.034	-0.912
224	0.085	0.526	104	-0.034	-4.926
106	0.084	1.969	136	-0.031	-1.309
107	0.084	1.749	195	-0.031	-1.415
241	0.084	2.805	223	-0.030	-1.172

TABLE 5. Top and worst Shapley values for MasterCard(MA).

Notable is that the sub-score from the linear model above decomposes as:

$$[\beta_1(R_m - R_f), \beta_2 \cdot SMB, \beta_3 \cdot HML] = [\beta_1, \beta_2, \beta_3]^T \cdot [(R_m - R_f), SMB, HML].$$

For a linear model the sub-score is just the macro coefficients times the fundamental data. Our model has a much more complex distribution between the independent variables and the target variable returns. As a result, there is not a uniform way to decompose our sub-scores into something more elementary and intrinsically linked to the model itself analogously as the β_i are linked to the linear model. This is a trade-off of the improved prediction power of our model versus the transparency of the linear model. However, we argue that the Shapley values provide a good insight into the black box predictions of a machine learning model and are a good substitute to the β_i .

In [LL], the authors identify Shapley values to the LIME method (Local Interpretable Model-agnostic Explanations) [GRS]. LIME locally approximates the model around a given prediction score. The local approximation is done by a linear model, and their conclusions are that Shapley values truly are “local linear” versions of the global β_i seen above. In other words, for each score, we can build a local linear model to get a simpler interpretation of the complex global model and the associated predictions are built additively from the initial features’s predictions.

5.3. Portfolio Forecasts. Rather than just state for our model the rank ordering power via Gini, or prediction error via R^2 , we show the economic prediction power of our model in this section. In particular, we create various portfolios and compare their risk-adjusted returns via the Sharpe Ratio.

ID	Top_Shapley_20	Std	ID	Worst_Shapley_20	Std
102	0.263	1.861	2	-0.093	-1.163
14	0.119	1.348	126	-0.068	-6.121
249	0.110	1.484	189	-0.065	-0.541
240	0.096	2.029	15	-0.065	-0.795
183	0.087	9.254	11	-0.063	-0.371
66	0.080	1.434	237	-0.053	-6.137
171	0.075	2.374	12	-0.049	-0.714
29	0.069	1.876	155	-0.048	-0.774
120	0.067	0.106	226	-0.048	-0.383
114	0.066	1.836	190	-0.044	-0.668
251	0.064	1.500	202	-0.044	-0.652
32	0.063	0.200	36	-0.044	-1.305
185	0.061	6.518	142	-0.042	-1.302
241	0.059	1.946	128	-0.041	-1.963
63	0.057	0.732	19	-0.038	-1.351
119	0.057	2.113	230	-0.038	-1.441
131	0.056	4.895	26	-0.037	-0.955
228	0.055	0.319	20	-0.034	-0.770
220	0.054	0.170	21	-0.033	-0.792
111	0.054	1.635	132	-0.032	-5.509

TABLE 6. Top and worst Shapley values for Goldman Sachs(GS).

ID	Top_Shapley_20	Std	ID	Worst_Shapley_20	Std
168	0.343	2.392	197	-0.055	-0.847
189	0.241	2.217	178	-0.055	-4.247
15	0.216	1.652	102	-0.053	-0.414
226	0.216	2.452	113	-0.052	-4.786
191	0.207	5.870	155	-0.050	-0.783
224	0.184	2.854	116	-0.049	-6.870
232	0.169	3.958	236	-0.049	-4.421
117	0.146	7.683	136	-0.045	-1.872
190	0.144	1.809	182	-0.044	-3.723
23	0.134	1.761	11	-0.043	1.048
2	0.129	0.681	176	-0.043	-9.303
106	0.127	1.367	105	-0.041	-2.778
20	0.125	1.560	32	-0.040	-0.653
202	0.122	2.016	5	-0.038	-1.637
49	0.118	3.235	121	-0.038	-8.963
220	0.114	0.978	80	-0.037	-1.479
12	0.109	1.243	238	-0.035	-0.893
123	0.106	2.565	254	-0.035	-2.710
157	0.101	5.977	234	-0.034	-3.024
231	0.093	2.433	217	-0.034	-3.271

TABLE 7. Top and worst Shapley values for Netflix(NFLX).

ID	Top_Shapley_20	Std	ID	Worst_Shapley_20	Std
159	0.137	1.971	187	-0.078	-4.726
2	0.130	0.733	11	-0.067	-0.712
32	0.109	0.643	108	-0.057	-2.743
12	0.107	1.339	134	-0.056	-5.377
20	0.100	1.206	252	-0.048	-0.965
247	0.092	1.536	102	-0.048	-0.364
163	0.083	4.344	15	-0.044	-0.493
26	0.081	0.730	214	-0.042	-1.811
220	0.080	1.041	84	-0.039	-2.011
120	0.079	0.981	230	-0.038	-1.459
28	0.079	1.566	253	-0.037	-2.519
202	0.076	1.289	36	-0.035	-0.138
49	0.074	2.437	250	-0.035	-2.520
228	0.072	0.808	6	-0.034	-1.015
133	0.070	1.039	66	-0.034	-0.807
25	0.069	1.485	105	-0.033	-2.728
14	0.069	0.077	217	-0.033	-3.242
41	0.065	1.206	164	-0.032	-1.052
218	0.063	1.672	125	-0.032	-2.482
175	0.062	1.134	216	-0.031	-2.807

TABLE 8. Top and worst Shapley values for Apple(AAPL).

ID	Top_Shapley_20	Std	ID	Worst_Shapley_20	Std
32	0.248	2.133	2	-0.102	-1.232
224	0.090	1.401	51	-0.089	-2.576
238	0.056	1.849	243	-0.089	-3.548
255	0.054	1.020	197	-0.088	-1.137
220	0.052	0.366	190	-0.078	-0.924
136	0.050	0.866	73	-0.077	-4.781
101	0.046	2.187	239	-0.076	-1.592
29	0.046	0.419	102	-0.072	-0.540
63	0.046	0.390	12	-0.071	-0.906
193	0.045	2.794	189	-0.069	-0.589
245	0.044	4.153	11	-0.066	-0.648
127	0.043	4.533	222	-0.065	-2.408
118	0.042	3.505	252	-0.061	-1.197
144	0.041	1.023	202	-0.061	-0.919
188	0.040	3.830	15	-0.060	-0.733
225	0.038	2.444	151	-0.057	-6.027
180	0.038	2.618	126	-0.055	-4.890
150	0.038	2.400	196	-0.054	-1.439
219	0.037	0.389	164	-0.052	-1.603
173	0.037	3.440	155	-0.051	-0.742

TABLE 9. Top and worst Shapley values for AT&T(T).

ID	Top_Shapley_20	Std	ID	Worst_Shapley_20	Std
133	0.113	2.169	189	-0.081	-0.583
209	0.105	5.747	102	-0.081	-0.611
154	0.098	5.959	2	-0.064	-0.924
255	0.084	2.435	200	-0.063	-1.013
211	0.062	4.622	11	-0.058	-0.073
197	0.061	0.721	155	-0.057	-0.849
144	0.060	1.799	15	-0.052	-0.627
196	0.059	2.066	12	-0.051	-0.728
129	0.057	3.536	111	-0.047	-1.249
227	0.057	3.108	32	-0.045	-0.691
29	0.054	0.735	125	-0.044	-3.554
246	0.054	1.490	164	-0.044	-1.342
169	0.046	5.334	23	-0.044	-0.900
152	0.044	5.913	190	-0.043	-0.809
14	0.043	-0.098	36	-0.042	-1.042
242	0.041	1.967	106	-0.037	-0.941
124	0.041	9.176	226	-0.037	-0.332
150	0.040	2.600	168	-0.035	-0.615
204	0.040	2.545	20	-0.034	-0.811
145	0.038	4.430	38	-0.033	-1.694

TABLE 10. Top and worst Shapley values for Wal-Mart(WMT).

We have many different ways of building our portfolio including varying the number of equities inside or the weight of each equity. We can also limit the sector weightings. We create individual tables below for each holdout month (Tables 11 - 15). Within each table, we create long only portfolios of sizes $\{10, 20, 50, 100\}$ with three different weight schemes. To check whether we focus too much on a single sector, we examine portfolios with less than four per sector or with no restriction at all. Again, sector is defined by Damodaran's sectors.

When we limit the number in our portfolio to X , we take the top X -equities ordered by their highest predicted model score.

With respect to our weighting schemes, we have three. The first is just equal weighting and is self explanatory. For the other two, recall our target variable is return adjusted by standard deviation. We create two weightings from these: score weighted and predicted return weighted. For a fixed number X in a portfolio, the score weighting is its associated model score divided by the sum of the other scores in the portfolio. The predicted return weighted setup is just the capital appreciation term from (4.1). We get it multiplying by the standard deviation of the returns of the specific equity during the training time period and add: *ten year bond yield - dividend yield*. Or we have:

$$\text{predicted return weight} := (\text{std of the equity return}) * (\text{model score}) + (\text{ten year bond yield} - \text{dividend yield}).$$

To help the reader understand the portfolio that got a score weighted Sharpe Ratio above 10 in Table 15 for *Apr19*, we show the twenty stocks as well as their model scores in that specific portfolio in Table 16. The portfolio is selected January 1, 2018 and held until April 30, 2019. Another question the reader may ask about this portfolio is: how does it decompose with respect to risk premium factors?

Num in Portfolio	10	20	50	100
S&P	-0.98	-0.98	-0.98	-0.98
Predicted return weight	-1.81	0.17	-0.03	-0.58
Score weighted	-2.46	-1.85	-0.98	-1.29
Equal Weight	-2.34	-1.76	-0.90	-1.27
Less than 4 of a sector				
Predicted return weight	-1.81	-2.51	-1.10	-0.58
Score weighted	-2.46	-2.15	-1.85	-1.73
Equal Weight	-2.34	-2.07	-1.78	-1.68

TABLE 11. Sharpe Ratio Dec 18.

Num in Portfolio	10	20	50	100
S&P	-0.44	-0.44	-0.44	-0.44
Predicted return weight	-0.26	0.38	1.07	1.12
Score weighted	1.53	0.92	1.51	1.82
Equal Weight	1.46	0.85	1.50	1.84
Less than 4 of a sector				
Predicted return weight	-0.26	0.38	-0.58	
Score weighted	1.53	1.08	0.96	2.03
Equal Weight	1.46	1.02	0.91	1.97

TABLE 12. Sharpe Ratio Jan 19.

5.3.1. *Risk Factor Regression.* To answer the prior question, we loaded our equal weighted portfolio into Portfolio Visualizer [PV] with the same dates as our holding period (1/1/18-4/30/19) using AQR’s six factor model. In other words, we use Fama-French’s Four Factor model plus AQR’s extra features explained below: QMJ and BAB. The value factor HML is swapped out by AQR’s HML-DEV. We mentioned this factor in Remark 3.1. For completion, we cite for the reader the risk premiums used:

- (1) HML-DEV, updated HML: High Minus Low: The value premium [AF]
- (2) MOM (also known as: UMD), Up Minus Down: The momentum premium, [FF1]
- (3) QMJ: Quality Minus Junk factor [AFP]
- (4) BAB: Betting against Beta [FP]

We notice the significant factors are from the Market (“Rm-Rf”), Size (“SMB”), and the unaccounted for Alpha (“Intercept”).

Num in Portfolio	10	20	50	100
S&P	-0.28	-0.28	-0.28	-0.28
Predicted return weight	0.92	0.62	3.94	10.21
Score weighted	3.69	3.76	5.61	5.66
Equal Weight	3.65	3.73	5.75	5.70
Less than 4 of a sector				
Predicted return weight	0.92	0.55	13.29	
Score weighted	3.69	3.57	5.23	3.08
Equal Weight	3.65	3.51	5.32	2.64

TABLE 13. Sharpe Ratio Feb 19.

Num in Portfolio	10	20	50	100
S&P	-0.19	-0.19	-0.19	-0.19
Predicted return weight	2.09	4.15	6.48	1.02
Score weighted	4.81	5.41	6.31	4.67
Equal Weight	4.73	5.41	6.38	4.52
Less than 4 of a sector				
Predicted return weight	2.09	4.15	2.27	0.72
Score weighted	4.81	5.03	3.63	2.61
Equal Weight	4.73	4.99	3.47	2.30

TABLE 14. Sharpe Ratio Mar 19.

Num in Portfolio	10	20	50	100
S&P	-0.05	-0.05	-0.05	-0.05
Predicted return weight	4.24	5.86	7.60	4.27
Score weighted	7.27	10.76	8.52	6.34
Equal Weight	7.22	10.81	8.41	6.13
Less than 4 of a sector				
Predicted return weight	4.24	5.86	0.70	3.57
Score weighted	7.27	8.45	5.16	4.36
Equal Weight	7.22	8.46	4.85	4.01

TABLE 15. Sharpe Ratio Apr 19.

Symbol	End April 2019	price_last_month_of_year_4	score
MA	245.61	151.36	7.17
V	161.02	114.02	7.12
FB	193.26	176.46	6.84
ISRG	517.78	364.94	6.79
NFLX	368.33	191.96	6.67
ADBE	277.66	175.24	6.61
SPGI	219.20	169.40	6.60
MCO	193.23	147.61	6.50
MTCH	62.17	31.31	6.46
NTNX	43.01	35.28	6.38
ELOX	12.16	8.00	6.32
ALGN	296.74	222.19	6.20
NIHD	1.93	0.42	6.16
EW	177.22	112.71	6.14
CRM	163.09	102.23	6.05
VKTX	8.35	4.06	6.05
KLAC	125.41	105.07	6.00
VSM	52.00	37.85	6.00
GOOG	1263.45	1046.40	5.82
PYPL	110.22	73.62	5.79

TABLE 16. Apr 19, Top 20 in portfolio.

Rm-Rf	SMB	HML-DEV	MOM	QMJ	BAB	Intercept	Annual Alpha	R^2
1.26	1.44	0.83	1.32	0.1	-0.77	3.86	46.33%	86.7%
(4.45)	(2.84)	(0.87)	(1.72)	(0.13)	(-0.84)	(3.32)		

TABLE 17. Four Factor Model with extra AQR features for Apr 19, Top 20 in portfolio.

5.4. Risks. There are several risks with our current modeling setup.

Data. Unlike other papers using machine learning to make a prediction such as [DGK], we are limited in time periods for our data. Therefore, the R^2 or Sharpe Ratios we see in Section 5 may not hold up if we used more historic data. Specifically, we may see as in Table 3 there is a larger drop off in Gini as one tries to predict one year ahead by comparing the Gini in-sample versus the holdout data. Our guess is that if for each year we used the same modeling setup of taking 4 years prior and creating trended data, we would see near the same performance. If, however, we concatenated twenty or thirty years of historic data into one train set for a single model, we would expect performance degradation.

Overfitting. Machine learning models are very good at overfitting training data. By that we mean a trained overfit model can be myopic to the returns of the train data with little ability to generalize predicting returns out of sample. Again, Table 3 says we retain good rank ordering even looking at data a year out.

However, it is possible that a shift in macro/market behavior can cause this rank ordering for out-of-sample data to severely decrease. To defend against such behavior, our recommendation is to update the model periodically and inspect the trending out of sample Gini. For these models to be applied well requires the data that the model predicts on to come from a similar distribution from the data it trained on. If a rolling Gini statistic seems out of the ordinary, precaution should be taken in using the model until the statistic stabilizes again. One would have to make rigorous what “similar” and “stabilizes” mean in a true investing atmosphere.

6. CURRENT FORECASTS

In this section, we discuss how to make predictions from today going forward. We remind the reader in Section 3 we discussed where we got our data as well as how we normalized it. Let us take as our goal making a prediction from the close of 7/29/19, for example. We train a model as before with the same analogous Sharpe ratio target variable dated as of today. To ensure we take at least a year from the target variable, we take the independent training data no later than 1/1/18. This gives us 19 months between the independent and dependent variables.

For the holdout data, we have analogous data as in prior sections but we make updates. For Morningstar data, we take the field “TTM” if we can obtain it. By that we mean, if this field is in the income/cash-flow sheets to use it. To update the balance sheet, we use the rolling quarterly updates of the balance sheet from Yahoo. This is not ideal but the numbers are comparable on annual balance sheet data. Up to these last updates to Morningstars’ TTM and Yahoo’s rolling update, we get within a quarter of an equities current accounting data.

With daily pricing/volume updates from Yahoo, we engineer the same features associated with our preprocessing in Section 3. Any feature that uses the Market Cap as a feature uses Market Cap as of the close price today.

To summarize, we normalize independent variables by data no later than 1/1/18 for the training data, and normalize by today’s date for the holdout data. Note: we have no target variable for the holdout data; the role of the model is to only rank order equities from today going forward 19 months.

Remark. It is possible to reduce the time window from 19 months to something smaller if we are retaining rolling quarterly accounting data. However, even if we could grab it for last year or the year before, to make a fair comparison, we would need all rolling quarters

going back four years. Unfortunately, at this time, none of this is publicly available from our stated data sources.

7. CONCLUSION

We wanted to show in this paper that with freely available data a retail investor could achieve a respectable Sharpe Ratio. Secondly, we wanted to show some of the interpretability of XGBoost on predicting asset returns.

On the former, we showed how one could grab data from Morningstar, Yahoo, Blackrock's Ishare, and Professor Damodaran's website, join them together, and process them into something suitable for supervised learning. We engineered many different features for our model including ones seen on Gurufocus as well as various Discounted Cash Flow features. Some of these constructed features are predictive and some are not. Our goal was just to collect them and evaluate performance from a rank ordering and economic perspective.

We showed with the above data, the gradient boosting machine learning model XGBoost could be used to obtain reasonable Sharpe Ratios on holdout data with a target variable of normalized returns dated more than a year out. We explained the macro importance variable Gain and how it allows us to see a feature's importance at a macro viewpoint. The viewpoint has a more refined micro viewpoint using Shapley values. We explained that Shapley values, which originated in game theory, were realized to be connected to interpreting Machine Learning models and later implemented in XGBoost. The values allow an interpretation of what drives the model prediction at the level of the equity itself. We explained these two ideas will be nearly the same for linear regression, but are different for the complex distribution of our training data with the normalized return target variable.

There are many more available data sources than these for the retail investor to investigate for predicting asset returns. We consider this paper as just a starting point.

7.1. Appendix.

Table 18: Appendix: ID to Feature

ID	Feature
1	E_divided_by_D_plus_E_4
2	Expected_growth_next_5_years_4
3	D_divided_by_D_plus_E_4
4	Expected_growth_next_5_years_average_last_4_years_from_4
5	Expected_growth_next_5_years_3
6	Reinvestment_Rate_4
7	PEG_Ratio_average_last_3_years_from_4
8	Research_and_development_EV_normalized_average_last_4_years_from_4
9	PEG_Ratio_3
10	price_last_month_of_year_4
11	Market_Cap_4
12	Research_and_development_EV_normalized_average_last_3_years_from_3
13	price_last_month_of_year_average_last_4_years_from_4
14	EBV_1
15	Total_assets_MC_normalized_3
16	Enterprise_value_4
17	PEG_Ratio_4
18	price_last_month_of_year_average_last_3_years_from_4
19	D_divided_by_D_plus_E_3
20	Research_and_development_Rev_normalized_average_last_4_years_from_4
21	Research_and_development_EV_normalized_3
22	Cash_and_cash_equivalents_4
23	Research_and_development_Rev_normalized_average_last_3_years_from_4
24	Market_Cap_average_last_3_years_from_4
25	Research_and_development_Rev_normalized_average_last_3_years_from_3
26	Total_assets_EV_normalized_3
27	Total_assets_EV_normalized_4
28	Total_assets_EV_normalized_average_last_4_years_from_4
29	Cash_and_cash_equivalents_2
30	DCF_damo_sector_growth_10Y_damo_growth_c_10_decay_ebit_10_ROIC_without_leases_20_per_share_pct_change_last_1_years_from_2
31	EV_Invested_Capital_pct_change_last_1_years_from_3
32	Expected_growth_next_5_years_pct_change_last_1_years_from_3
33	Altman_Z_score_average_last_4_years_from_4
34	Gross_property_plant_and_equipment_EV_normalized_3
35	DCF_morn_YOY_damo_growth_c_5_decay_ebit_75_roic_80_4

36	Enterprise_value_3	
37	Gross_property_plant_and_equipment_EV_normalized_average_last_3_years_from_3	
38	Market_Cap_3	
39	Days_Payable_4	
40	Tangible_book_value_average_last_4_years_from_4	
41	cash_not_needed_for_operations_4	
42	Total_assets_MC_normalized_4	
43	Accounts_payable_Rev_normalized_4	
44	Gross_property_plant_and_equipment_EV_normalized_2	
45	Net_income_4	
46	Total_current_assets_EV_normalized_2	
47	volume_months_back_3	
48	EV_over_FCF_pct_change_last_1_years_from_3	
49	Deferred_revenues_4	
50	Free_Cash_Flow_USD_Mil_average_last_4_years_from_4	
51	Depreciation_and_amortization_Rev_normalized_4	
52	Net_Income_USD_Mil_4	
53	volume_months_back_9	
54	DCF_damo_sector_growth_10Y_damo_growth_c.2_decay_ebit_50_ROIC_with_leases_60_per_share_pct_change_last_1_years_from_4	
55	volume_months_back_9_sector_std	
56	Gross_property_plant_and_equipment_EV_normalized_average_last_3_years_from_4	
57	PEG_Ratio_1	
58	DCF_damo_sector_growth_3Y_damo_growth_c.15_decay_ebit_30_ROIC_with_leases_5_per_share_average_last_3_years_from_4	
59	Altman_Z_score_average_last_3_years_from_4	
60	volume_months_back_1	
61	Operating_income_MC_normalized_4	
62	ROC_Greenblatt_gross_PPE_average_last_4_years_from_4	
63	Cash_and_cash_equivalents_average_last_4_years_from_4	
64	Research_and_development_MC_normalized_average_last_4_years_from_4	
65	Earnings_Per_Share_USD_average_last_3_years_from_4	
66	volume_months_back_18_sector_std	
67	Number_of_firms_4	
68	Book_Value_Per_Share_USD_1	
69	Cash_and_cash_equivalents_average_last_3_years_from_4	
70	Three_stage_DCF_morn_3Y_scratch_growth_c.2.7_decay_ebit_80_30_roic_70_20_per_share_4	
71	Gross_property_plant_and_equipment_EV_normalized_average_last_4_years_from_4	
72	Operating_income_EV_normalized_average_last_3_years_from_4	
73	Pre_tax_Operating_Margin_after_lease_adj_pct_change_last_1_years_from_3	
74	BV_of_Equity_4	

75	Acc_Rec_Sales_average_last_4_years_from_4	
76	BV_of_Equity_2	
77	Common_stock_issued_2	
78	Income_before_taxes_average_last_3_years_from_4	
79	volume_months_back_6	
80	Gross_Margin_4	
81	BV_of_Capital_1	
82	volume_months_back_12	
83	DCF_morn_YOY_scratch_growth.c.15_decay_ebit_40_roic_with_SGA_amor_25	
84	Accounts_payable_Rev_normalized_2	
85	Altman_Z_score_pct_change_last_1_years_from_4	
86	Change_in_working_capital_EV_normalized_pct_change_last_1_years_from_4	
87	DCF_morn_YOY_damo_growth.c.5_decay_ebit_50_roic_60_pct_change_last_1_years_from_3	
88	Dividend_paid_4	
89	Other_current_assets_pct_change_last_1_years_from_3	
90	Current_PE_pct_change_last_1_years_from_4	
91	Diluted_Earnings_per_share_average_last_3_years_from_3	
92	DCF_damo_sector_growth_10Y_damo_growth.c.10_decay_ebit_50_ROIC_without_leases_60_4	
93	Inventories_EV_normalized_average_last_4_years_from_4	
94	Current_PE_average_last_3_years_from_3	
95	DCF_damo_sector_growth_10Y_scratch_growth.c.15_decay_ebit_40_ROIC_with_leases_25_per_share_pct_change_last_1_years_from_3	
96	Three_stage_DCF_morn_10Y_damo_growth.c.5_12_decay_ebit_60_10_roic_50_10_4	
97	DCF_damo_sector_growth_3Y_scratch_growth.c.5_decay_ebit_50_roic_with_RD_amor_60	
98	Capital_expenditure_EV_normalized_2	
99	DCF_morn_YOY_damo_growth.c.5_decay_ebit_75_roic_80_per_share_4	
100	Acc_Rec_Sales_pct_change_last_1_years_from_3	
101	DCF_morn_3Y_damo_growth.c.15_decay_ebit_40_roic_25_3	
102	volume_months_back_12_sector_std	
103	DCF_damo_sector_growth_10Y_scratch_growth.c.15_decay_ebit_40_roic_25_pct_change_last_1_years_from_3	
104	DCF_damo_sector_growth_3Y_damo_growth.c.10_decay_ebit_30_roic_30_per_share_average_last_3_years_from_3	
105	Change_in_Accounts_receivable_pct_change_last_1_years_from_4	
106	D_divided_by_D_plus_E_average_last_3_years_from_4	
107	E_divided_by_D_plus_E_average_last_3_years_from_4	
108	Days_Payable_1	
109	Income_before_taxes_average_last_3_years_from_3	
110	Receivables_Rev_normalized_4	
111	Net_Margin_pct_change_last_1_years_from_3	
112	DCF_morn_5Y_damo_growth.c.10_decay_ebit_10_roic_20_pct_change_last_1_years_from_4	
113	Three_stage_damo_sector_growth_10Y_scratch_growth.c.5_12_decay_ebit_60_10_ROIC_with_leases_50_10_per_share_pct_change_last_1_years_from_4	

```

114 cash_not_needed_for_operations_3
115 cash_not_needed_for_operations_2
116 Change_in_working_capital_MC_normalized_3
117 DCF_damo_sector_growth_3Y_scratch_growth.c.15_decay_ebit_40_ROIC_without_leases_25_pct_change_last_1_years_from_4
118 DCF_morn_10Y_damo_growth.c.5_decay_ebit_75_roic_80_3
119 Purchases_of_investments_4
120 Stock_based_compensation_average_last_3_years_from_3
121 Three_stage_damo_sector_growth_10Y_damo_growth.c.2_7_decay_ebit_80_30_roic_70_20_per_share_pct_change_last_1_years_from_2
122 EBV_per_share_1
123 Stock_based_compensation_4
124 Other_long_term_liabilities_EV_normalized_pct_change_last_1_years_from_2
125 DCF_damo_sector_growth_10Y_damo_growth.c.15_decay_ebit_40_roic_with_SGA_amor_25
126 Accumulated_other_comprehensive_income_average_last_4_years_from_4
127 Intangible_assets_MC_normalized_pct_change_last_1_years_from_3
128 EV_over_EBITDA_4
129 DCF_damo_sector_growth_3Y_scratch_growth.c.15_decay_ebit_30_roic_with_SGA_amor_5
130 Payout_Ratio_average_last_3_years_from_4
131 Intangible_assets_Rev_normalized_average_last_4_years_from_4
132 Long_term_debt_repayment_pct_change_last_1_years_from_4
133 DCF_damo_sector_growth_10Y_scratch_growth.c.5_decay_ebit_10_ROIC_with_leases_10_3
134 DCF_morn_5Y_scratch_growth.c.15_decay_ebit_40_roic_25_4
135 DCF_morn_YOY_damo_growth_2
136 PBV_pct_change_last_1_years_from_3
137 Net_Income_USD_Mil_pct_change_last_1_years_from_3
138 Tax_Rate_average_last_3_years_from_3
139 Return_on_Assets_pct_change_last_1_years_from_3
140 Return_on_Assets_pct_change_last_1_years_from_4
141 Investments_gains_losses_1
142 price_months_back_1
143 price_months_back_3
144 DCF_damo_sector_growth_3Y_damo_growth.c.10_decay_ebit_10_roic_with_RD_amor_20
145 Effect_of_exchange_rate_changes_pct_change_last_1_years_from_4
146 D_divided_by_D_plus_E_pct_change_last_1_years_from_4
147 Research_and_development_MC_normalized_3
148 Intangible_assets_average_last_3_years_from_3
149 DCF_damo_sector_growth_per_share_1
150 DCF_damo_sector_growth_5Y_damo_growth.c.10_decay_ebit_10_roic_with_RD_amor_20
151 Accumulated_other_comprehensive_income_4
152 positive_non_operating_cash_average_last_3_years_from_3

```

153 Three.stage.damo.sector.growth_3Y_scratch.growth.c.5.10_decay_ebit_70_40_ROIC_with_leases.60_30_pct.change.last.1.years.from.3
154 positive_non_operating_cash_average.last.3.years.from.4
155 Expected_growth_next.5.years_average.last.3.years.from.4
156 Other_financing_activities_pct.change.last.1.years.from.4
157 DCF_damo.sector.growth_YOY_scratch.growth.c.15_decay_ebit_40_ROIC_without_leases.25_pct.change.last.1.years.from.4
158 Operating_income_Rev_normalized_pct.change.last.1.years.from.4
159 Total_assets_EV_normalized_average.last.3.years.from.4
160 Payout_Ratio.3
161 Acquisitions_and_dispositions_average.last.3.years.from.3
162 Gross_Margin.3
163 DCF_morn.5Y_scratch.growth.c.10_decay_ebit_50_roic.60.2
164 price.months.back.12.sector.mean
165 Three.stage.damo.sector.growth_3Y_scratch.growth.c.5.10_decay_ebit_70_40_ROIC_with_leases.60_30_per_share_pct.change.last.1.years.from.3
166 Current_Ratio_pct.change.last.1.years.from.2
167 ROC_Greenblatt.gross.PPE.1
168 Total_assets_MC_normalized_average.last.3.years.from.4
169 DCF_damo.sector.growth_10Y_scratch.growth.c.15_decay_ebit_40_roic.25.3
170 PE_ratio_pct.change.last.1.years.from.4
171 Other_operating_activities.4
172 E.divided.by.D.plus.E_pct.change.last.1.years.from.3
173 DCF_damo.sector.growth_3Y_scratch.growth.c.5_decay_ebit_10_ROIC_with_leases.10.4
174 Net_Margin_pct.change.last.1.years.from.4
175 Research_and_development_EV_normalized.4
176 Three.stage.damo.sector.growth_3Y_damo.growth.c.2.7_decay_ebit_80_30_roic_70_20_per_share_pct.change.last.1.years.from.2
177 Market_Debt_to_Capital_with_leases_pct.change.last.1.years.from.4
178 DCF_damo.sector.growth_YOY_scratch.growth.c.10_decay_ebit_30_ROIC_with_leases.30_pct.change.last.1.years.from.2
179 DCF_damo.sector.growth_5Y_damo.growth.c.15_decay_ebit_40_ROIC_without_leases.25_per_share_average.last.3.years.from.3
180 DCF_damo.sector.growth_10Y_damo.growth.c.15_decay_ebit_40_ROIC_without_leases.25_per_share_pct.change.last.1.years.from.3
181 DCF_morn_YOY_scratch.growth_per_share_average.last.3.years.from.3
182 volume.months.back.12.sector.mean
183 Receivables_EV_normalized.4
184 Effect_of_exchange_rate_changes.2
185 Other_assets_and_liabilities.2
186 Prepaid_expenses.1
187 Three.stage.DCF_morn.10Y_scratch.growth.c.2.10_decay_ebit_90_20_roic_80_10_per_share_average.last.3.years.from.3
188 DCF_morn_YOY_damo.growth.c.15_decay_ebit_30_roic_with_SGA_amor.5
189 EV_Invested.Capital.1
190 EV_Invested.Capital.3
191 Change_in_working_capital_EV_normalized.1

192 Other_investing_activities_pct_change_last_1_years_from_4
 193 DCF_damo_sector_growth_5Y_scratch_growth.c.5_decay_ebit_10_roic_with_RD_amor_10
 194 Inventories_pct_change_last_1_years_from_2
 195 DCF_morn_YOY_damo_growth_per_share_2
 196 Free_Cash_Flow_USD_Mil_pct_change_last_1_years_from_4
 197 Revenue_growth_Three_Year_Average_pct_change_last_1_years_from_2
 198 Cash_at_beginning_of_period_average_last_3_years_from_3
 199 cash_not_needed_for_operations_average_last_3_years_from_4
 200 Dividends_USD_pct_change_last_1_years_from_4
 201 DCF_damo_sector_growth_10Y_scratch_growth.c.5_decay_ebit_10_ROIC_with_leases_10_4
 202 Debt_repayment_pct_change_last_1_years_from_3
 203 cash_not_needed_for_operations_average_last_3_years_from_3
 204 DCF_damo_sector_growth_10Y_scratch_growth.c.5_decay_ebit_10_roic_with_RD_amor_10
 205 DCF_morn_10Y_damo_growth_pct_change_last_1_years_from_3
 206 Depreciation_and_amortization_EV_normalized_pct_change_last_1_years_from_3
 207 Other_working_capital_3
 208 Total_current_liabilities_pct_change_last_1_years_from_2
 209 positive_non_operating_cash_average_last_4_years_from_4
 210 Research_and_development_average_last_3_years_from_3
 211 Effect_of_exchange_rate_changes_4
 212 Free_Cash_Flow_USD_Mil_4
 213 Total_assets_EV_normalized_2
 214 Equity_EVA_pct_change_last_1_years_from_3
 215 Pre_tax_Operating_Margin_after_lease_adj_pct_change_last_1_years_from_4
 216 DCF_morn_5Y_scratch_growth.c.10_decay_ebit_10_roic_20_per_share_average_last_3_years_from_3
 217 Beta_1
 218 positive_non_operating_cash_3
 219 positive_non_operating_cash_2
 220 positive_non_operating_cash_4
 221 DCF_morn_YOY_scratch_growth.c.15_decay_ebit_30_roic_5_per_share_2
 222 price_months_back_9_sector_mean
 223 E_divided_by_D_plus_E_3
 224 Tangible_book_to_Market_4
 225 DCF_morn_10Y_damo_growth_average_last_3_years_from_3
 226 Aggregate_Mkt_Cap_Net_Income_3
 227 Other_current_assets_pct_change_last_1_years_from_2
 228 Stock_based_compensation_average_last_4_years_from_4
 229 Acquisitions_and_dispositions_3
 230 Intangible_assets_average_last_3_years_from_4

```

231 Forward_PE_2
232 Research_and_development_MC_normalized_average_last_3_years_from_3
233 DCF_morn_3Y_damo_growth_c_15_decay_ebit_30_roic_with_RD_amor_5
234 DCF_damo_sector_growth_10Y_scratch_growth_c_10_decay_ebit_30_ROIC_with_leases_30_pct_change_last_1_years_from_2
235 Receivables_Rev_normalized_1
236 Three_stage_damo_sector_growth_3Y_scratch_growth_c_5_12_decay_ebit_60_10_ROIC_with_leases_50_10_per_share_pct_change_last_1_years_from_4
237 Pre_tax_Operating_Margin_before_lease_adj_pct_change_last_1_years_from_4
238 Current_PE_2
239 Intangible_assets_4
240 Property_and_equipments_net_average_last_3_years_from_3
241 Property_and_equipments_net_average_last_3_years_from_4
242 DCF_morn_10Y_damo_growth_c_15_decay_ebit_30_roic_with_SGA_amor_5
243 Price_Operating_Income_PE_ratio_divided_by_growth_pct_change_last_1_years_from_2
244 Total_assets_Rev_normalized_1
245 PEG_Ratio_pct_change_last_1_years_from_3
246 Cash_at_end_of_period_2
247 Working_Capital_USD_Mil_2
248 Effect_of_exchange_rate_changes_average_last_3_years_from_4
249 Property_and_equipments_net_1
250 Net_income_available_to_common_shareholders_pct_change_last_1_years_from_3
251 Property_and_equipments_net_average_last_4_years_from_4
252 Deferred_revenues_pct_change_last_1_years_from_4
253 ROE_average_last_3_years_from_3
254 Additional_paid_in_capital_pct_change_last_1_years_from_4
255 Net_income_3

```

Feature	Score	Feature	Score
MA	7.170	PFE	3.755
V	7.117	MMM	3.720
FB	6.838	MS	3.610
ISRG	6.792	MO	3.588
NFLX	6.673	GS	3.579
ADBE	6.614	SBUX	3.567
SPGI	6.597	AMT	3.449
CRM	6.049	PM	3.404
GOOG	5.825	CAT	3.383
PYPL	5.792	TJX	3.259
UNH	5.754	UNP	3.247
INTU	5.707	LMT	3.108
GOOGL	5.682	CMCSA	3.096
AMZN	5.651	AXP	3.088
TXN	5.604	CVS	3.078
NVDA	5.349	HON	3.038
BKNG	5.170	ADP	3.026
BLK	5.150	LOW	2.992
SYK	5.039	AVGO	2.975
CME	5.025	KO	2.971
NKE	4.909	UPS	2.839
ANTM	4.896	NEE	2.822
LLY	4.671	BAC	2.737
MDT	4.656	UTX	2.703
AAPL	4.610	COST	2.697
BSX	4.564	CL	2.626
MCD	4.433	CSX	2.602
BMJ	4.432	IBM	2.497
BDX	4.411	PEP	2.420
ABBV	4.403	CB	2.331
CELG	4.392	CVX	2.250
TMO	4.364	C	2.179
ABT	4.333	USB	2.179
MSFT	4.302	XOM	2.167
CSCO	4.282	JPM	2.093
JNJ	4.260	DIS	2.023
HD	4.237	DUK	1.926
DHR	4.187	VZ	1.881
GILD	4.181	WMT	1.865
AMGN	4.087	SO	1.817
BA	4.074	CHTR	1.769
CI	4.056	PG	1.732
ORCL	4.041	D	1.696
MRK	4.038	CCI	1.597
ACN	3.953	MDLZ	1.562
QCOM	3.815	PNC	1.536
INTC	3.808	WFC	1.529
PFE	3.755	COP	1.500
MMM	3.720	T	1.300
MS	3.610	GE	0.901

TABLE 19. Large cap stocks used for Shapley standard deviation.

REFERENCES

- [AF] *The Devil in HMLs Details*, Clifford Asness and Andrea Frazzini
- [AFP] *Quality Minus Junk* Cliff Asness, Andrea Frazzini, Lasse H. Pedersen Review of Accounting Studies March 2019, Volume 24, Issue 1, pp 34112 Cliff Asness, Andrea Frazzini, Lasse H. Pedersen
- [AUC] <https://en.wikipedia.org/wiki/AUC>
- [Brei] L. Breiman. *Random forests*. Machine Learning, 45(1):532, Oct. 2001
- [Damod] Aswath Damodaran http://pages.stern.nyu.edu/~adamodar/New_Home_Page/dataarchived.html
- [DamorD] Aswath Damodaran *Research and Development Expenses: Implications for Profitability Measurement and Valuation*
- [DamoeVA] Aswath Damodaran http://people.stern.nyu.edu/adamodar/New_Home_Page/lectures/eva.html
- [DGK] Dacheng Xiu, Shihao Gu, Bryan Kelly *Empirical Asset Pricing via Machine Learning* Jun. 2018. In Revision.
- [ELL] Scott M. Lundberg, Gabriel G. Erion, Su-In Lee *Consistent Individualized Feature Attribution for Tree Ensembles* <https://arxiv.org/abs/1802.03888>
- [Euc] John Alberg, Zachary C. Lipton <https://arxiv.org/pdf/1711.04837.pdf>
- [FP] Andrea Frazzini, Lasse H. Pedersen *Betting Against Beta* Journal of Financial Economics Volume 111, Issue 1, January 2014, Pages 1-25
- [FF1] Fama, E. F. and French, K. R. (1993). *Common risk factors in the returns on stocks and bonds*. Journal of Financial Economics, 33(1):356.
- [FF2] *The CrossSection of Expected Stock Returns* Fama, E. F. and French, K. R., (1992) <https://doi.org/10.1111/j.1540-6261.1992.tb04398.x> Cited by: 3637
- [Gini] <https://www.kaggle.com/cpptz/gini-coefficient-an-explanation-with-math/>
- [GKX] Shihao Gu, Bryan Kelly, Dacheng Xiu *Autoencoder Asset Pricing Models*,
- [GRS] Marco Tulio Ribeiro, Sameer Singh, Carlos Guestrin "Why Should I Trust You?": Explaining the Predictions of Any Classifier
- [Guru] Gurufocus <https://www.gurufocus.com/>
- [HXZ] Kewei Hou, Chen Xue, Lu Zhang *Digesting Anomalies: An Investment Approach*
- [IJR] Ronen Israel, Sarah Jiang, and Adrienne Ross *Craftsmanship Alpha: An Application to Style Investing*
- [Ishares] iShares Russell 3000 ETF <https://www.ishares.com/us/products/239714/ishares-russell-3000-etf>
- [KM] https://en.wikipedia.org/wiki/K-means_clustering
- [LL] Scott M. Lundberg, Su-In Lee *A Unified Approach to Interpreting Model Predictions*, 31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.
- [Maubdcf] Michael Mauboussin https://www3.nd.edu/~scorwin/fin70610/Common%20DCF%20Errors_LeggMason.pdf
- [Morn] Morningstar <https://www.morningstar.com/>
- [M] Kevin P. Murphy *Machine Learning: A Probabilistic Perspective*
- [PV] <https://www.portfoliovisualizer.com>
- [Shapwiki] https://en.wikipedia.org/wiki/Shapley_value
- [StratCV] https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html
- [Tfid] https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfTransformer.html#sklearn.feature_extraction.text.TfidfTransformer
- [Valu] Tim Koller, Marc Goedhart, David Wessels *Valuation: Measuring and Managing the Value of Companies, 5th Edition* 5th Edition
- [XGB] Tianqi Chen, Carlos Guestrin *XGBoost: A Scalable Tree Boosting System* <https://arxiv.org/abs/1603.02754>
- [Yahoo] Yahoo Finance <https://finance.yahoo.com/>
- [Zoo] Dacheng Xiu, Guanhao Feng and Stefano Giglio, *Taming the Factor Zoo: a Test of New Factors* Apr. 2019.